

Desenvolvimento Web 1

Prof. Diego Max

Aula 08:

CSS3 – Posicionamento, Layout
e Responsividade



Tópicos desta aula

- Modelo de Caixa (Box model)
- Display: block, inline e inline-block
- Esquemas de Posicionamento (static, relative, absolute, fixed e sticky)
- Layout Flexbox
- Responsividade com media queries



Aula 08.1:

Modelo de Caixa (Box model)



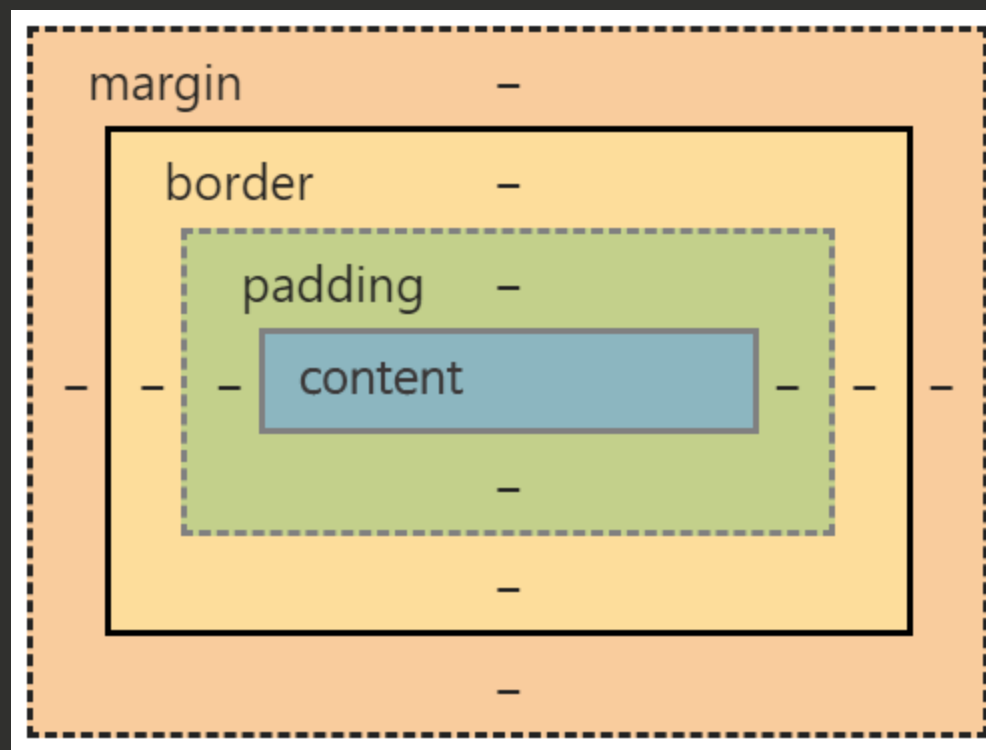
O que é o Modelo de Caixa (Box Model) no CSS?

No CSS, todo elemento da página é tratado como uma **caixa**. O modelo de caixa define como essa caixa é formada e como o navegador calcula o tamanho final de cada elemento na tela.

Essa caixa é composta por quatro partes, do centro para fora:

- Content
- Padding
- Border
- Margin

📄 O modelo de caixa define como o espaço de um elemento é construído, somando conteúdo (**content**), preenchimento (**padding**), borda (**border**) e margem (**margin**) ao tamanho total.



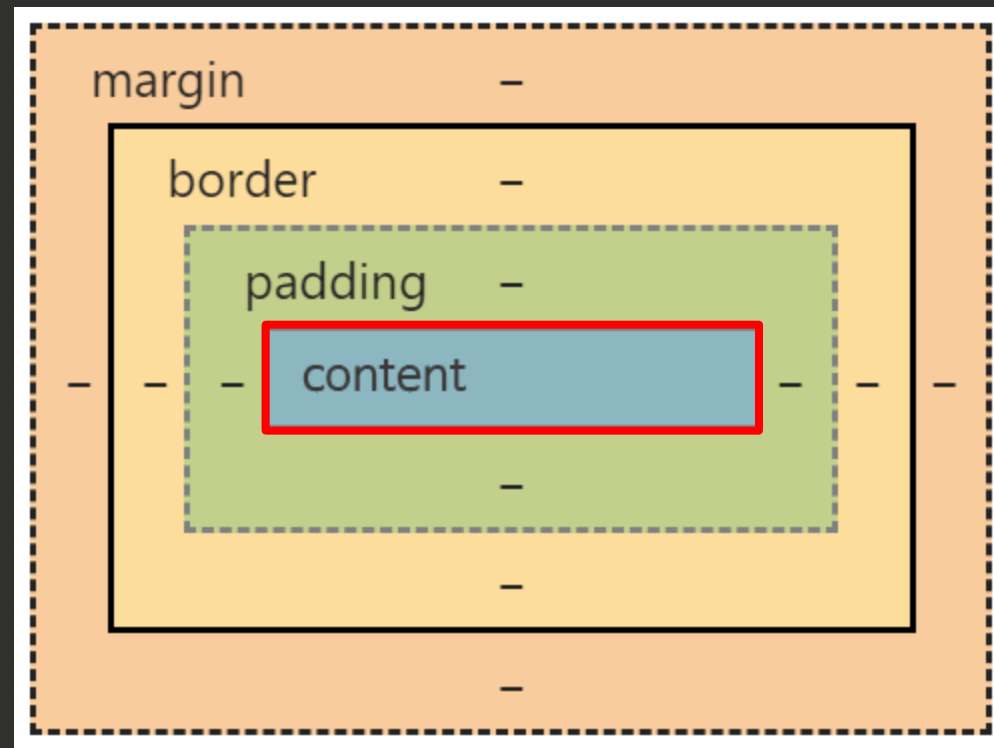
Content (Conteúdo)

É a parte central da caixa, onde fica:

- Texto
- Imagens
- Vídeos
- Outros elementos

As propriedades **width** e **height**, por padrão, se aplicam apenas ao conteúdo.

```
div {  
  width: 200px;  
  height: 100px;  
}
```

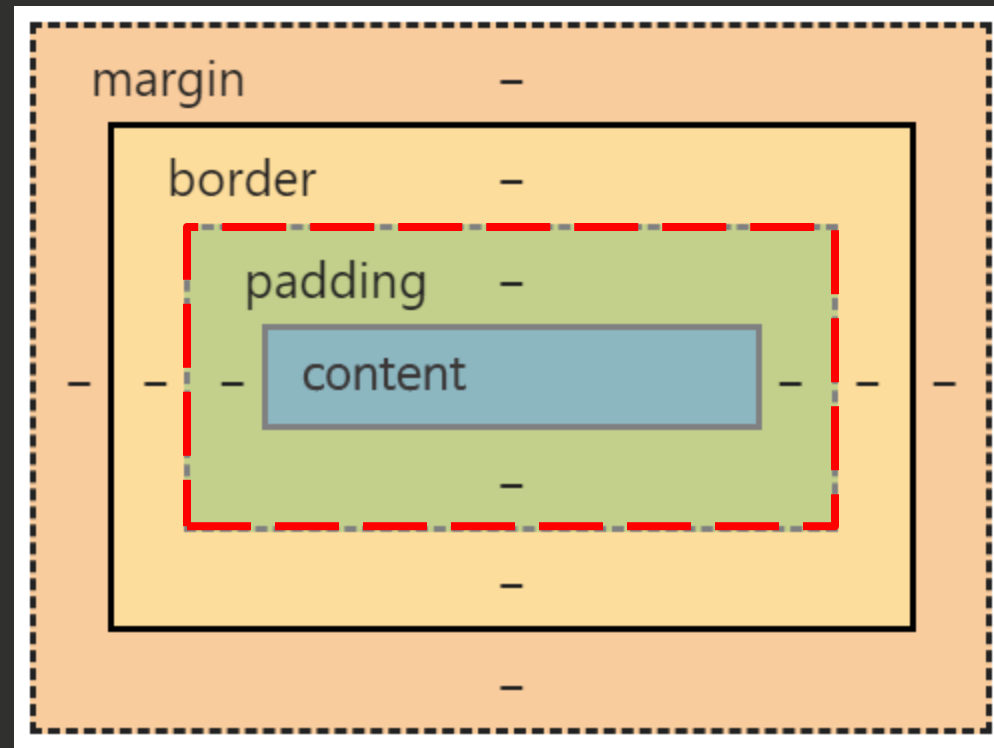


Padding (Preenchimento)

É o espaço interno, entre o conteúdo e a borda.

- Afasta o conteúdo da borda
- Aumenta visualmente o tamanho do elemento
- Recebe a cor de fundo (background-color)

```
div {  
    padding: 20px;  
}
```

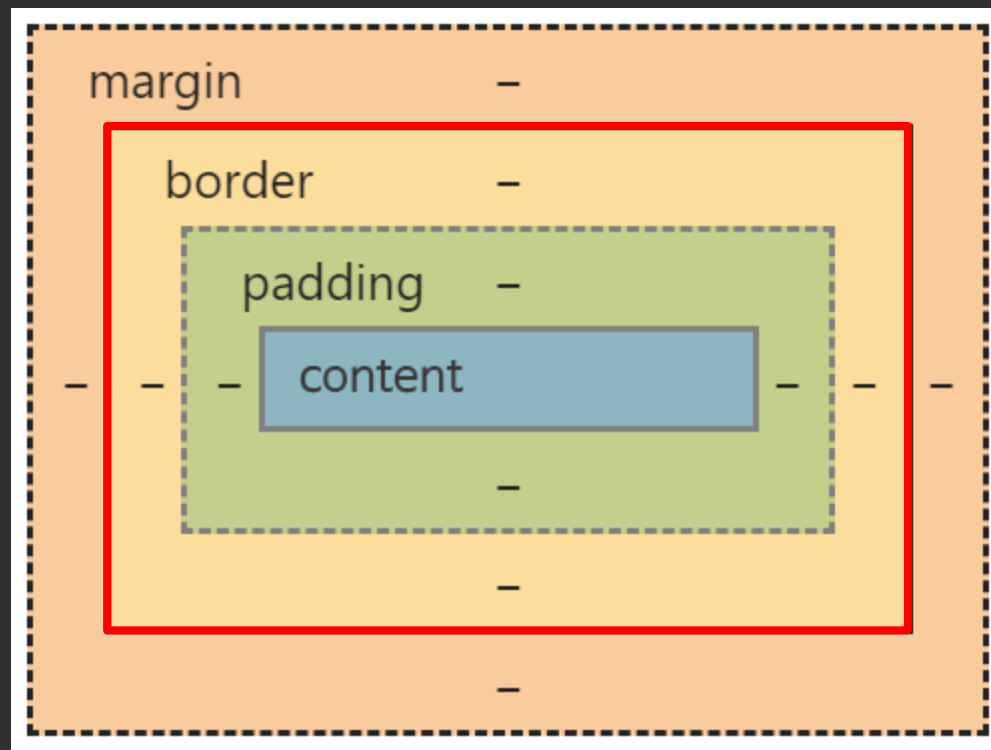


Border (Borda)

É a linha que envolve o padding e o conteúdo.

- Pode ter espessura, cor e estilo
- Também aumenta o tamanho final do elemento

```
div {  
  border: 2px solid black;  
}
```

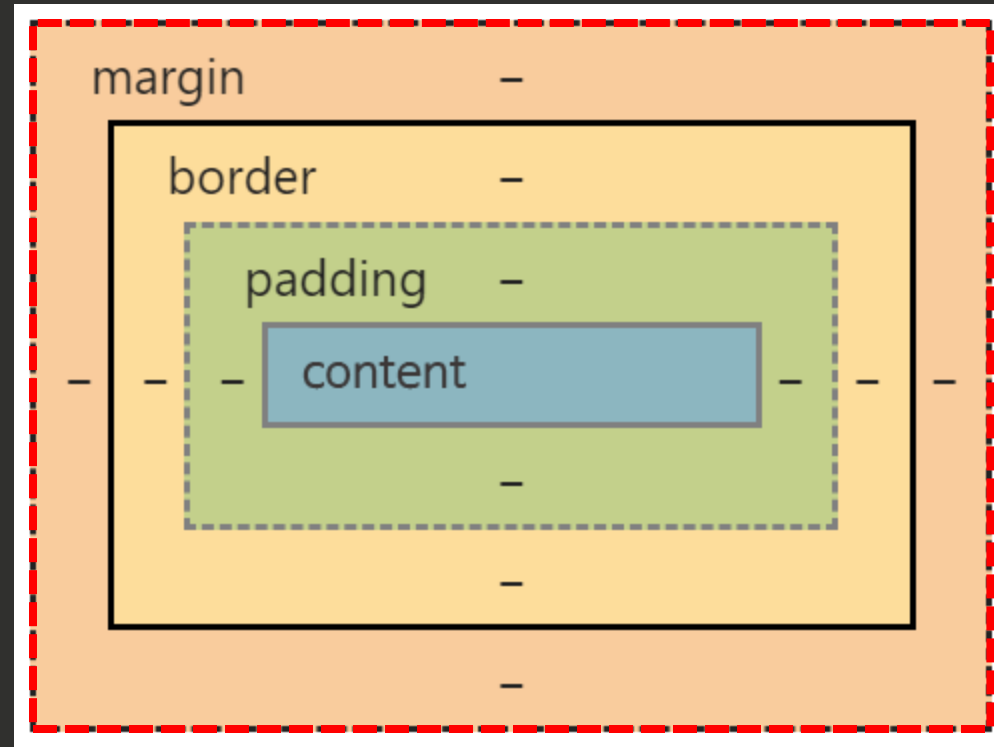


Margin (Margem)

É o espaço externo da caixa.

- Afasta um elemento dos outros
- Não recebe cor de fundo
- Não faz parte da caixa visível

```
div {  
  margin: 15px  
}
```



Como o navegador calcula o tamanho final de um elemento?

Por padrão os elementos recebem o valor **“content-box”** para a propriedade **“box-sizing”**, isso significa que as propriedades **width** e **height** definem apenas o tamanho do **conteúdo**, e que o **padding** e a **border** são somados a esse valor, **aumentando o tamanho final do elemento** na página. Mas isso pode ser alterado.

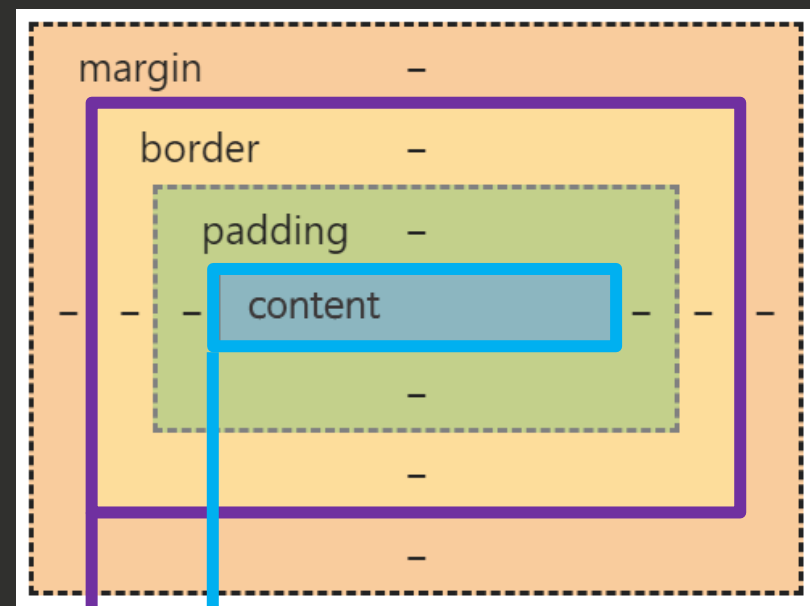
Para facilitar o layout, é comum usar:

```
* {  
  box-sizing: border-box;  
}
```

Nesse caso:

- width e height já incluem padding e border
- O tamanho do elemento fica mais previsível
- Muito usado em layouts modernos.

Em ambos os casos a **margin** não entra no cálculo da caixa, mas influencia o espaço ao redor.



Aula 08.3:

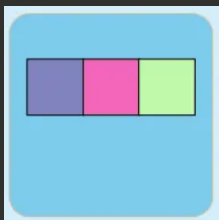
Esquemas de Posicionamento



Esquemas de Posicionamento

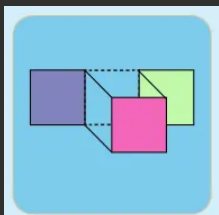


Position: Static e Relative



Static (Padrão): O posicionamento estático é o comportamento natural. Elementos seguem o fluxo normal do documento e propriedades `top`, `right`, `bottom`, `left` não têm efeito.

```
.elemento {  
  position: static;  
  /* Comportamento padrão */  
}
```



Relative: O elemento mantém seu espaço original no fluxo, mas pode ser deslocado visualmente usando `top`, `right`, `bottom`, `left`. Outros elementos não são afetados.

```
.elemento {  
  position: relative;  
  top: 20px;  
  left: 30px;  
}
```

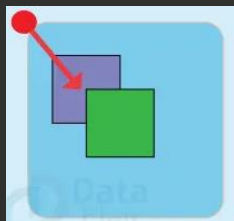


Caso de uso comum: Position **relative** é frequentemente usado como referência para elementos com position **absolute** dentro dele, criando um contexto de posicionamento.

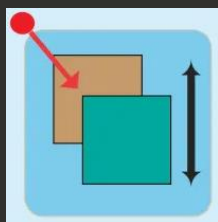
Esquemas de Posicionamento



Position: Absolute e Fixed



Absolute: Remove o elemento do fluxo normal. Ele é posicionado em relação ao ancestral posicionado mais próximo que não seja static (*geralmente um elemento com position relative*).



Fixed: Posiciona o elemento em relação à janela do navegador (viewport). Permanece fixo mesmo com rolagem da página.



Casos de uso comum :

Absolute: Ideal para modais e overlays, tooltips e dropdowns, elementos decorativos.

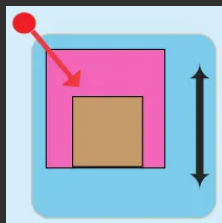
Fixed: Cabeçalhos e rodapés fixos, botões de ação flutuantes, barras de navegação persistentes.

```
.modal {  
  position: absolute;  
  top: 50%;  
  left: 50%;  
  transform: translate(-50%, -50%);  
}
```

```
.header {  
  position: fixed;  
  top: 0;  
  left: 0;  
  width: 100%;  
  z-index: 1000;  
}
```

Esquemas de Posicionamento

Position: Sticky



Sticky: O position: sticky é uma combinação híbrida de **relative** e **fixed**. O elemento se comporta como **relative** até que atinja um limite definido durante a rolagem, quando então "**gruda**" como fixed.

```
.sidebar-nav {  
  position: sticky;  
  top: 20px;  
  /* Gruda quando chega a  
  20px do topo */  
}
```

Requisitos Importantes

- Definir ao menos uma direção (top, bottom, left, right)
- O elemento pai deve ter altura suficiente
- Não funciona com overflow: hidden no pai



Casos de uso comum :

Stiky: Cabeçalhos de tabelas durante scroll,
Barras laterais de navegação,
Elementos que devem acompanhar o usuário.

Fixed vs Sticky



A propriedade z-index

A propriedade **z-index** controla a ordem de empilhamento dos elementos, ou seja, qual elemento fica na frente ou atrás quando eles se sobrepõem na página.

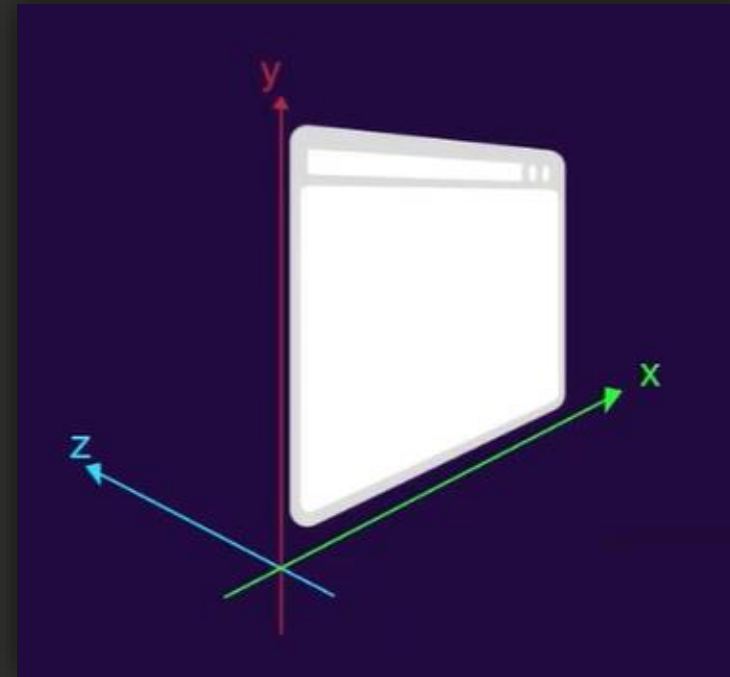
Pense no z-index como camadas, semelhantes às camadas de um editor de imagens.

Quando o z-index funciona?

O z-index só funciona em elementos que possuem uma posição definida, como:

- position: relative;
- position: absolute;
- position: fixed;
- position: sticky;

Não funciona com position: static (padrão).



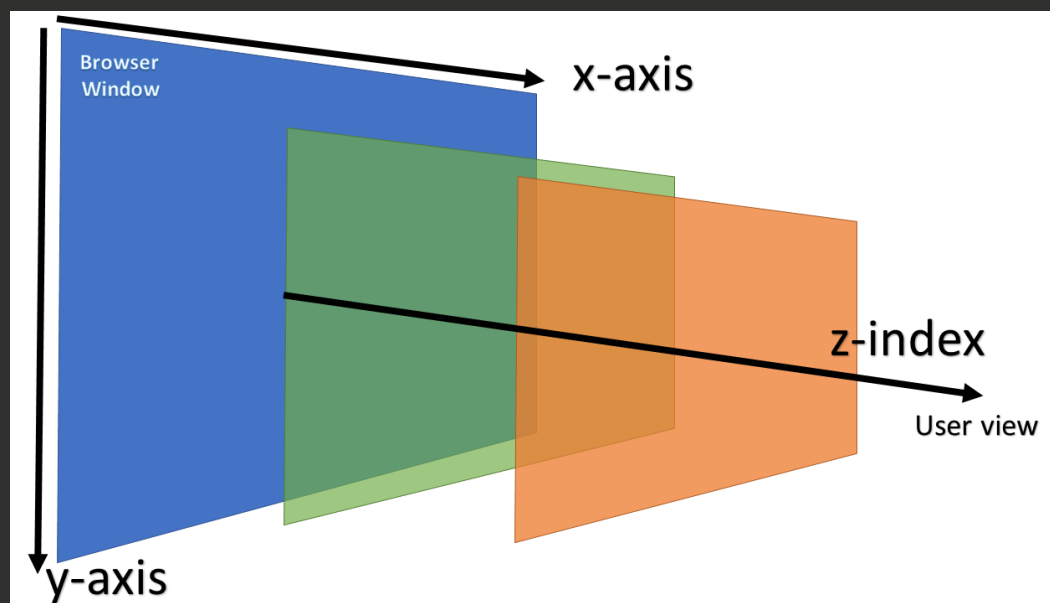
Como funciona o valor do z-index?

- Quanto maior o número, mais à frente o elemento fica.
- Quanto menor o número, mais ao fundo: static (padrão).

```
.caixa1 {  
  position: relative;  
  z-index: 1;  
}
```

```
.caixa2 {  
  position: relative;  
  z-index: 2;  
}
```

A **.caixa2** ficará sobre a **.caixa1**.



Aula 08.2:

Display: block, inline e inline-block



Display: block, inline e inline-block



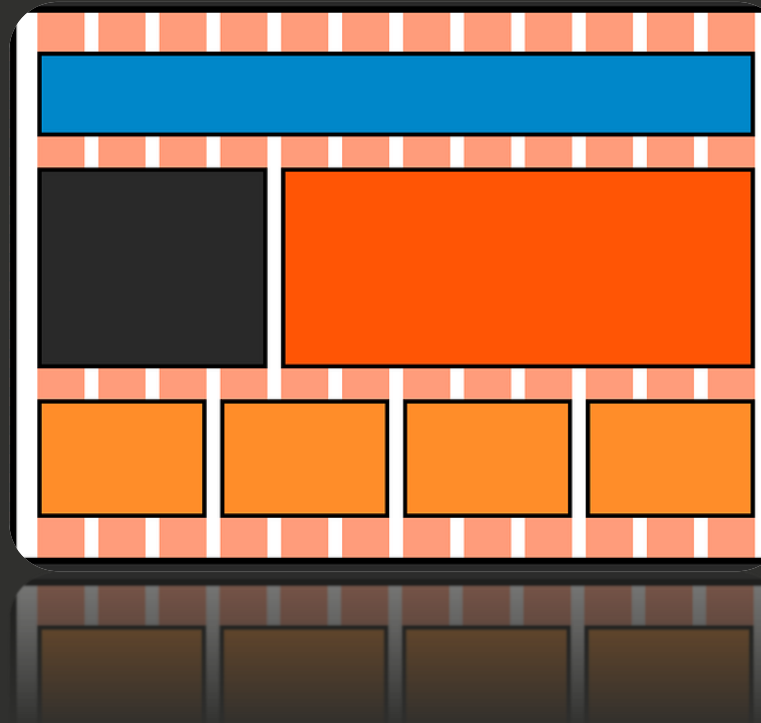
No HTML e no CSS, os elementos podem se comportar basicamente de três formas:

- **Elementos de bloco (block)**
- **Elementos em linha (inline)**
- **Elementos em linha-bloco (inline-block)**

Esse comportamento influencia como eles ocupam espaço na página e como se organizam no layout.

De forma geral:

- **Block** organiza a página
- **Inline** organiza o texto
- **Inline-block** organiza componentes visuais



Elementos de Bloco (display: block)



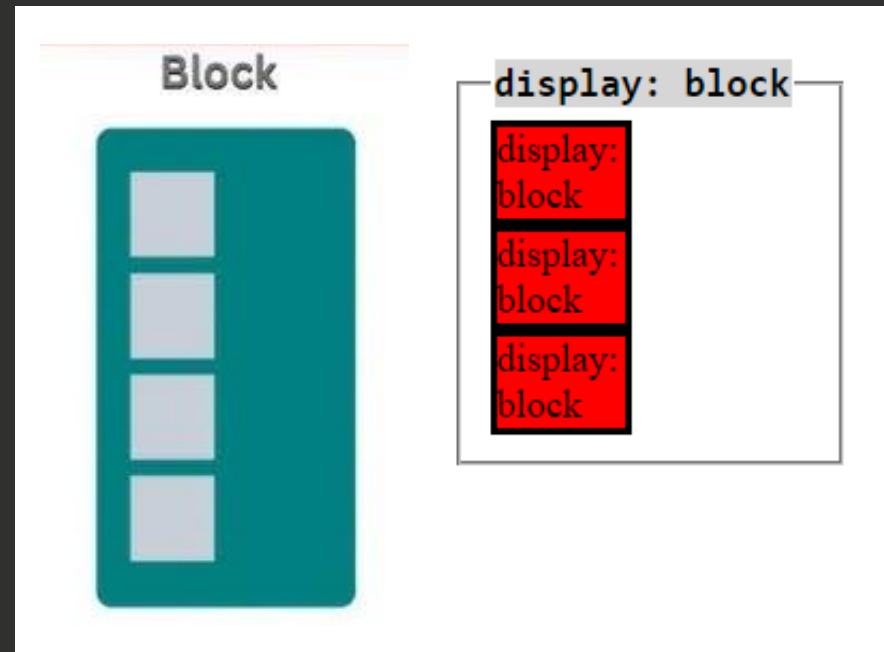
Como funcionam:

- Sempre começam em uma nova linha
- Ocupam toda a largura disponível do elemento pai
- Aceitam normalmente:
 - width e height
 - margin (top, right, bottom, left)
 - padding
- Empurram outros elementos para baixo

Exemplos comuns:

`<div>`, `<p>`, `<h1>` a `<h6>`, `<section>`,
`<article>`, `<header>`, `<footer>`

(Organizam a página)



Elementos em Linha (display: inline)



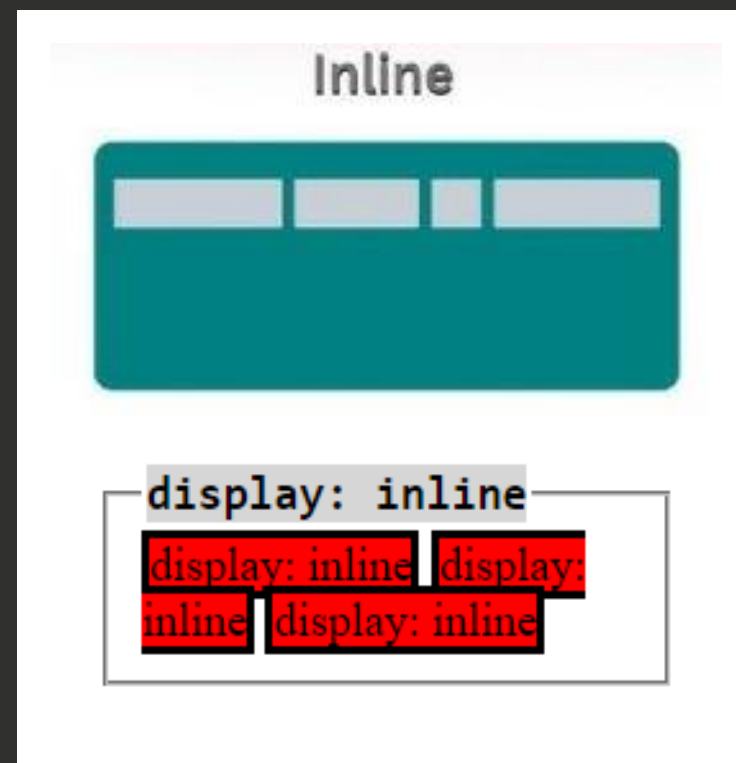
Como funcionam:

- Não quebram linha
- Ocupam apenas o espaço do conteúdo
- Fluem com o texto
- Não aceitam:
 - width e height
 - margin e padding(funcionam apenas horizontalmente)
- Usados principalmente dentro de textos

Exemplos comuns:

, <a>, , , <label>

(Organizam o texto)



Elementos Bloco em Linha (display: inline-block)



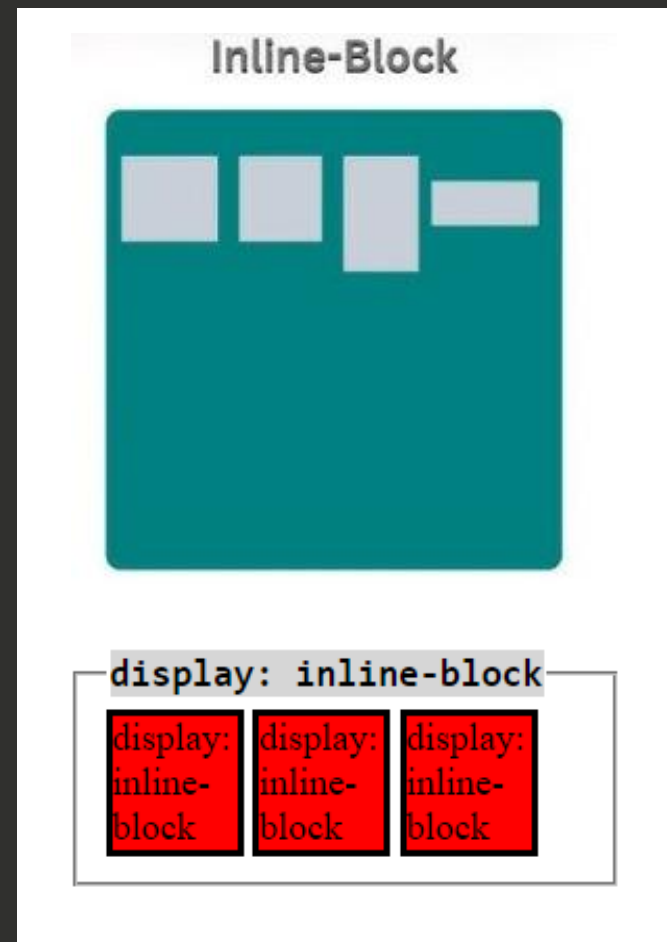
Como funcionam:

- Ficam na mesma linha (como inline)
- Mas aceitam:
 - width e height
 - margin e padding (como block)
- Não forçam quebra de linha
- Muito usados para criar cards, botões e menus horizontais.

Exemplos comuns:

`<img>`, `<button>`, `<input>`

(Organizam componentes visuais)



Aula 08.4:

Layout Flexbox

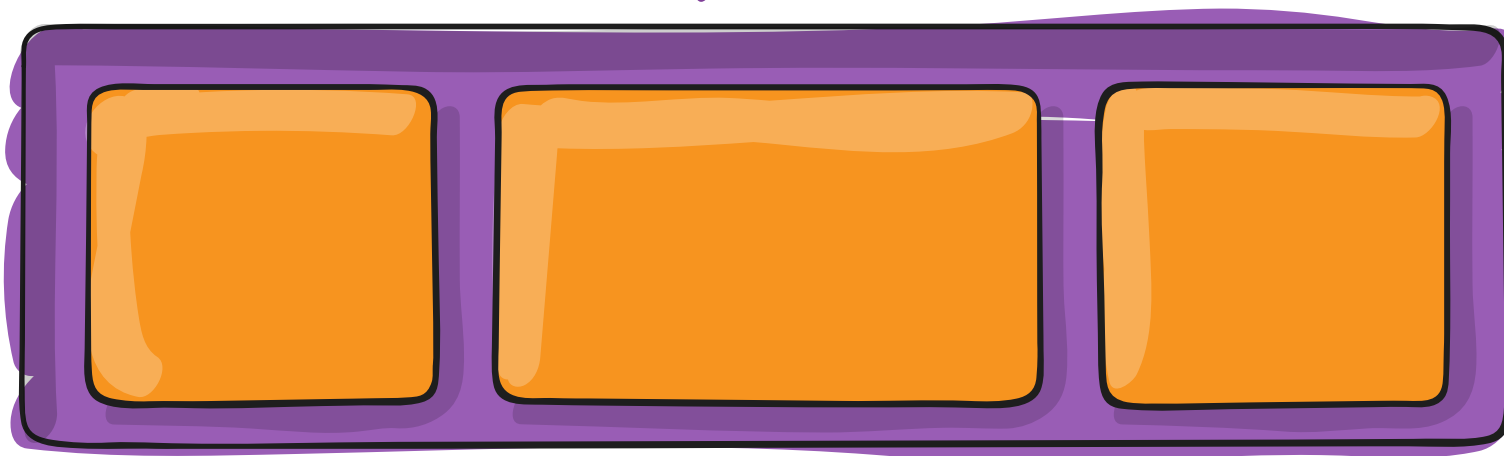


Flexbox: O Poder dos Layouts Unidimensionais

Flexbox é um modelo de layout unidimensional que distribui espaço e alinha itens em uma linha ou coluna. Revolucionou a forma como criamos layouts responsivos, eliminando hacks com *float* e *position*.



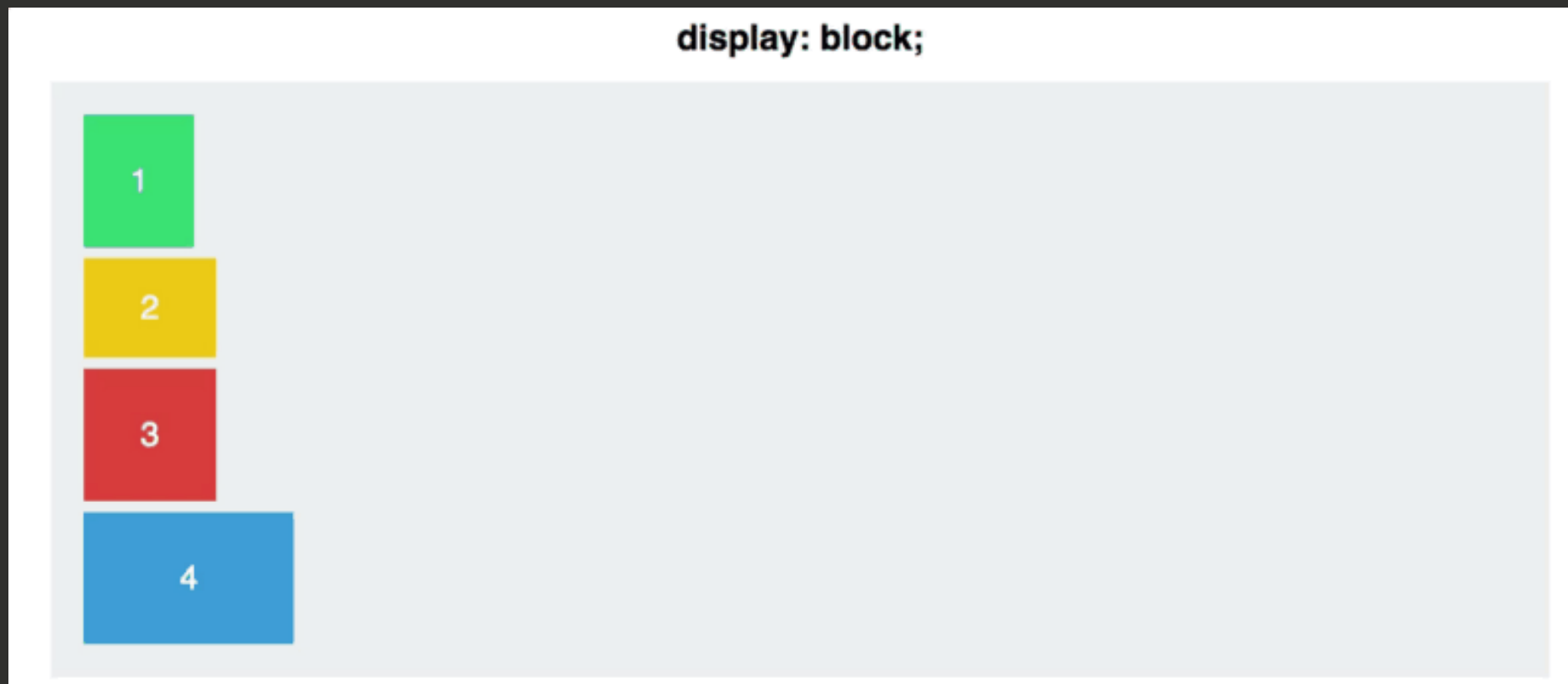
container



Flexbox: O Poder dos Layouts Unidimensionais

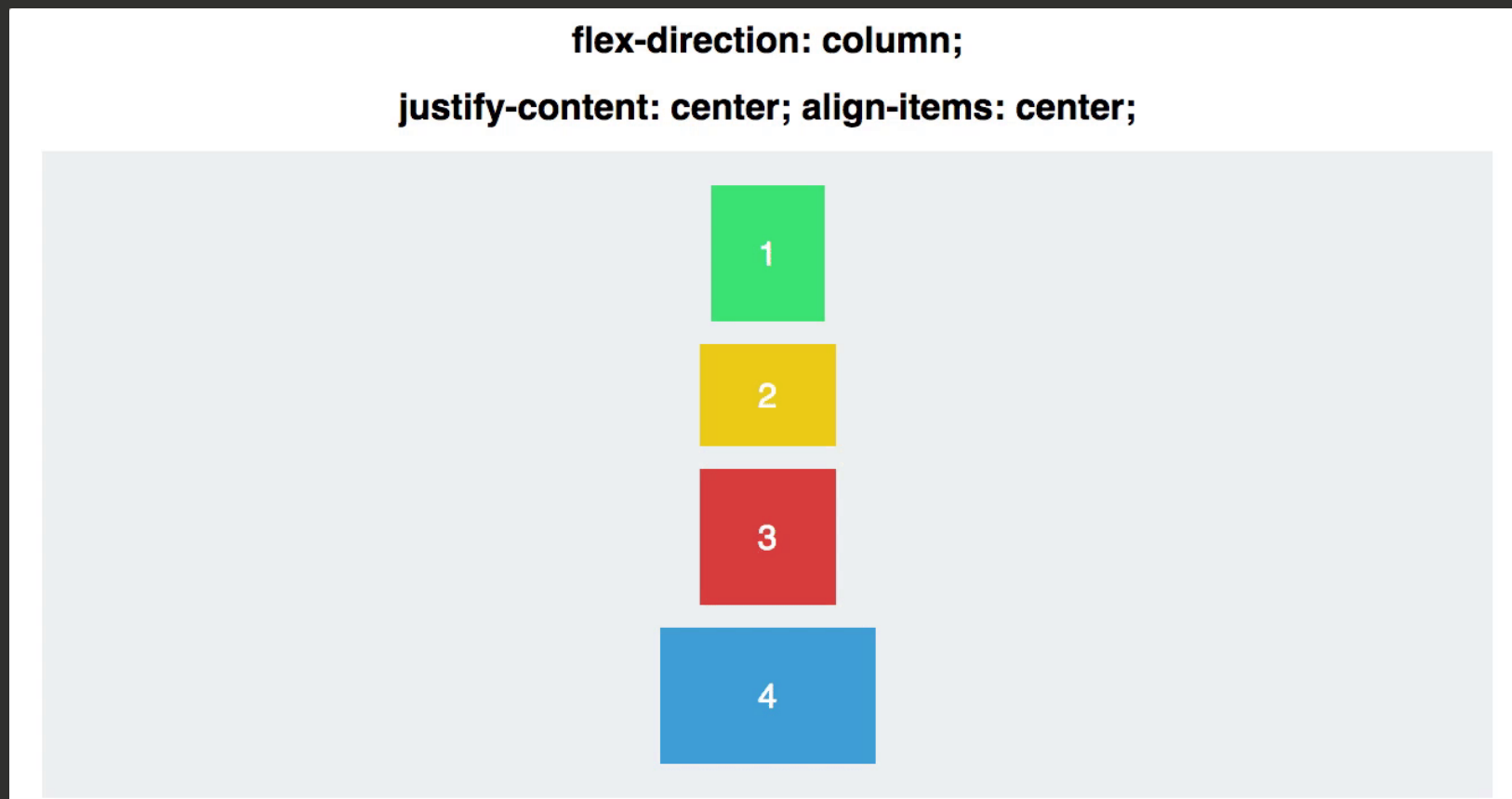
Transforme qualquer elemento em um **flex container** aplicando:

`display: flex` ou `display: inline-flex`.



Flexbox: O Poder dos Layouts Unidimensionais

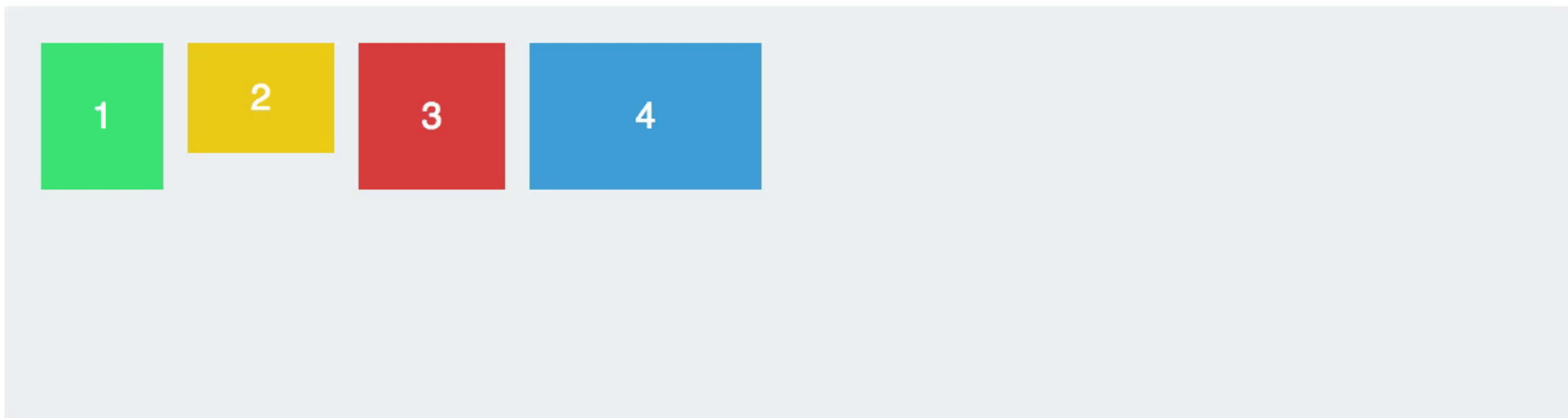
Use propriedades como **flex-direction**, **justify-content** e **align-items** para controlar o layout dos itens filhos.



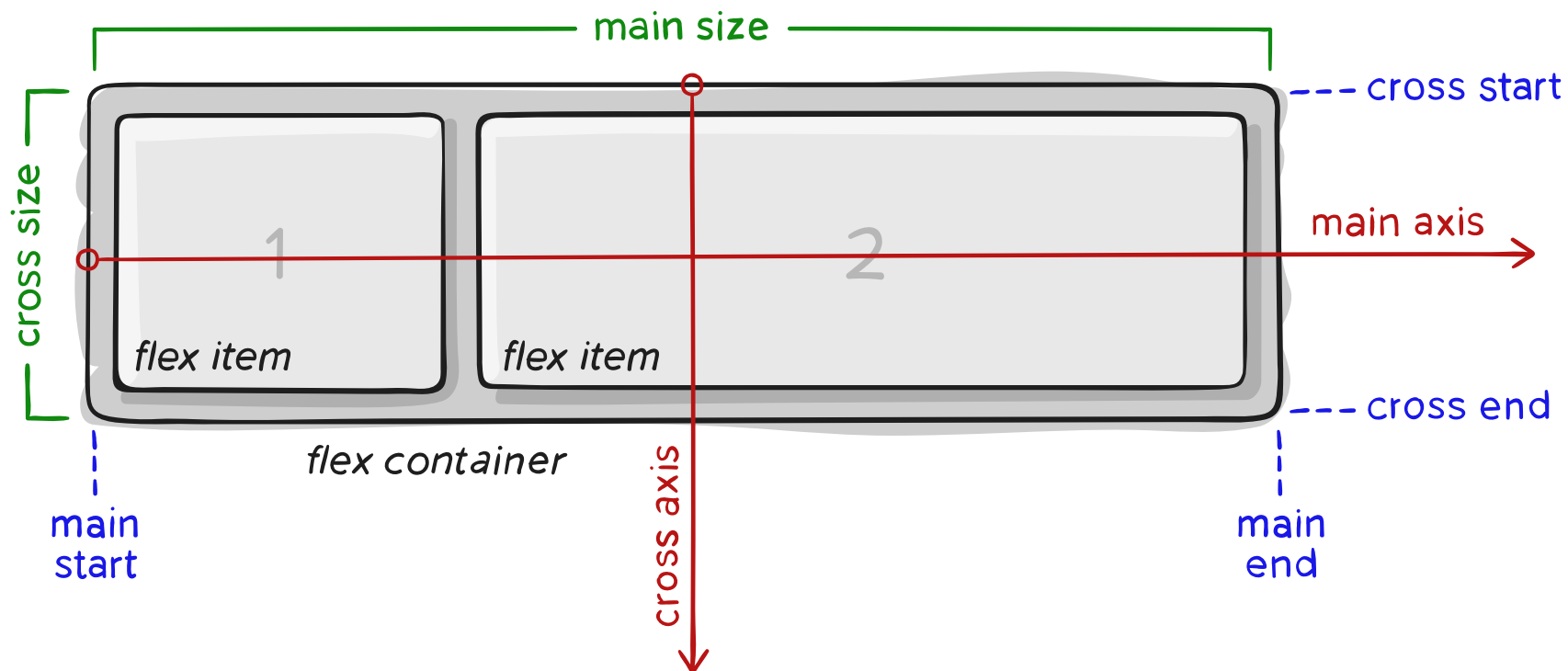
Flexbox: O Poder dos Layouts Unidimensionais

Use propriedades como **flex-direction**, **justify-content** e **align-items** para controlar o layout dos itens filhos.

align-items: flex-start;



Alinhando os itens do Flexbox



Alinhando os itens do Flexbox



No Flexbox, os elementos são organizados a partir de dois eixos:

Eixo principal (main axis): é o eixo definido pela propriedade **flex-direction**. Ele determina a direção em que os itens são posicionados (horizontal em **row** ou vertical em **column**). O alinhamento nesse eixo é controlado por **justify-content**.

Eixo cruzado (cross axis): é o eixo perpendicular ao eixo principal. Ele define como os itens são alinhados “de lado” em relação à direção principal. O alinhamento nesse eixo é controlado por **align-items** e **align-self**.

Em resumo: o **eixo principal** define o **fluxo dos itens** e o **eixo cruzado** define o **alinhamento lateral desses itens**.

Flexbox: O Poder dos Layouts Unidimensionais



Ative o Flexbox

Transforme qualquer elemento em um flex container aplicando `display: flex` ou `display: inline-flex`.



Configure o Container

Use propriedades como `flex-direction`, `justify-content` e `align-items` para controlar o layout dos itens filhos.



Ajuste os Itens

Propriedades como `flex-grow`, `flex-shrink` e `align-self` permitem controle individual sobre cada item flex.

Propriedades do Flex Container



1

flex-direction

Define o eixo principal do layout: `row` (padrão), `row-reverse`, `column`, `column-reverse`.

2

flex-wrap

Controla se os itens quebram em múltiplas linhas: `nowrap` (padrão), `wrap`, `wrap-reverse`.

3

justify-content

Alinha itens no eixo principal: `flex-start`, `center`, `flex-end`, `space-between`, `space-around`, `space-evenly`.

4

align-items

Alinha itens no eixo cruzado: `stretch` (padrão), `flex-start`, `center`, `flex-end`, `baseline`.

5

align-content

Distribui espaço entre linhas quando há wrap: `flex-start`, `center`, `space-between`, `stretch`.

6

gap

Define espaçamento entre itens: `gap: 20px` ou `gap: 10px 20px` (row column).

Propriedades dos Flex Items

order

Controla a ordem visual dos itens sem alterar o HTML. Padrão é 0, valores menores aparecem primeiro.

```
.item-1 { order: 2; }  
.item-2 { order: 1; }  
.item-3 { order: 3; }
```

flex-grow

Define quanto um item cresce em relação aos outros. Padrão é 0 (não cresce).

```
.item {  
  flex-grow: 1;  
  /* Cresce igualmente */  
}
```

flex-shrink

Define quanto um item encolhe quando não há espaço. Padrão é 1 (pode encolher).

```
.item {  
  flex-shrink: 0;  
  /* Não encolhe */  
}
```

flex-basis

Define o tamanho inicial do item antes da distribuição de espaço: `flex-basis: 200px` ou `flex-basis: 30%`.

flex (shorthand)

Combina grow, shrink e basis: `flex: 1 1 auto` ou simplesmente `flex: 1` para crescer igualmente.

align-self

Sobrescreve align-items para um item específico: `align-self: center` ou `align-self: flex-end`.

Aula 08.5:

Responsividade com media queries



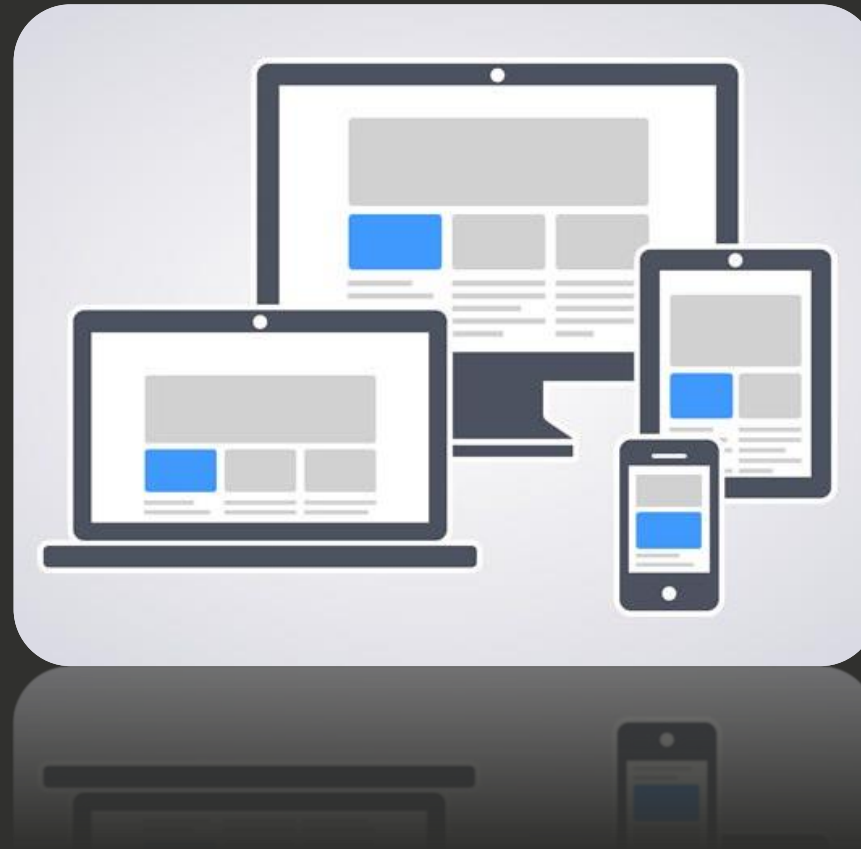
Layout responsivos com media queries

Uma **media query** é uma técnica CSS que permite aplicar estilos diferentes dependendo das características do dispositivo ou da janela do navegador (largura, altura, orientação, etc.).

É a base do design responsivo moderno.

Sintaxe Básica

```
@media (condição) {  
    /* Estilos que serão aplicados  
    quando a condição for verdadeira */  
}
```



Exemplo Prático

Largura máxima (Mobile):

```
@media (max-width: 768px) {  
  .Logo {  
    font-size: 1rem;  
  }  
  
  .cards {  
    flex-direction: column;  
    align-items: center;  
  }  
  
  .sobre-flex {  
    flex-direction: column;  
  }  
  
  .footer-content {  
    flex-direction: column;  
  }  
}
```

Aplica estilos quando a tela tem **até 768px de largura**

Nesse caso, quando a tela atingir essa largura os elementos que eram organizados em **linha** pelo **flex container** passam a se organizar em **colunas** para caberem na tela menor.

Breakpoints Comuns



Dispositivo	Largura	Media Query
Mobile	até 480px	(max-width: 480px)
Mobile grande	481px – 768px	(min-width: 481px) and (max-width: 768px)
Tablet	769px – 1024px	(min-width: 769px) and (max-width: 1024px)
Desktop	1025px+	(min-width: 1025px)

Material complementar



- **Guia do Flexbox - Alura:**

Disponível em: <https://www.alura.com.br/artigos/css-guia-do-flexbox>

- **Simulador de Flexbox:**

Disponível em: <https://www.treinaweb.com.br/blog/css-flexbox-um-guia-interativo-parte-1-containers>



Exercício de fixação - Aula 08:

<https://forms.office.com/r/rfBh9Pf7zC>

Desenvolvimento Web 1

Prof. Diego Max